

UNIVERSITY OF SCIENCE, VNUHCM
FACULTY OF INFORMATION TECHNOLOGY



SOFTWARE TESTING

HW04 – Automation Testing

STUDENT INFORMATION

Field	Detailed Information
Full Name	Nguyen An Nghiep
Student ID	23127234
Github	https://github.com/ntritrn999/CSC13003-EShop

Task 1

Feature selection

The three features selected in HW02 and automated in HW04 are:

Pool	Feature	Automated cases	Test-data file	Specification file
A	FR-03 - Forgot/Reset Password	17	playwright/test-data/fr03.csv	playwright/tests/fr03.password-reset.spec.ts
B	FR-10 - Order State Machine	13	playwright/test-data/fr10.csv	playwright/tests/fr10.order-state.spec.ts

Pool	Feature	Automated cases	Test-data file	Specification file
C	FR-15 - Product CRUD	23	playwright/test-data/fr15.csv	playwright/tests/fr15.product-crud.spec.ts

All test inputs and expected outcomes are stored in separate CSV files. The TypeScript specifications contain the reusable workflows, setup, cleanup, and assertions.

Pool A: FR-03 - Forgot/Reset Password

Test cases:

Test ID	Objective	Type	Preconditions	Input / strategy	Main steps	Expected result
TC_FR03_01	Reset a registered user's password with valid inputs.	Positive	test@eshop.com exists.	Displayed OTP; Test1234!	Request OTP, enter the displayed OTP and password, submit.	Success dialog and redirect to /login.
TC_FR03_02	Request a reset for an unregistered email.	Negative	Email is not registered.	missing-23127234@eshop.invalid	Open forgot-password form and submit the email.	User not found; reset step 2 is not opened.
TC_FR03_03	Reject an OTP shorter than six digits.	Negative / Boundary	Registered user; OTP requested.	OTP 12345; valid password.	Enter five-digit OTP and submit reset.	OTP/token rejection; remain on forgot-password page.
TC_FR03_04	Reject an incorrect six-digit OTP.	Negative	Registered user; OTP requested.	OTP 999999; valid password.	Enter incorrect OTP and submit reset.	OTP/token rejection; password is not reset.
TC_FR03_05	Reject a seven-character password.	Negative / Boundary	Registered user; OTP requested.	Displayed OTP; Aa1!bbb.	Enter valid OTP and short password, submit.	Password rejection.
TC_FR03_06	Reject a password without uppercase letters.	Negative	Registered user; OTP requested.	Displayed OTP; test1234!.	Enter password and submit.	Password rejection.
TC_FR03_07	Reject a password without	Negative	Registered user; OTP requested.	Displayed OTP; TEST1234!.	Enter password	Password rejection.

Test ID	Objective	Type	Preconditions	Input / strategy	Main steps	Expected result
	lowercase letters.				and submit.	
TC_FR03_08	Reject a password without a number.	Negative	Registered user; OTP requested.	Displayed OTP; TestPass! .	Enter password and submit.	Password rejection.
TC_FR03_09	Reject a password without a special character.	Negative	Registered user; OTP requested.	Displayed OTP; Test12345.	Enter password and submit.	Password rejection.
TC_FR03_10	Verify that a confirmation-password field is provided.	Negative / UI	Registered user; OTP requested.	Inspect reset step 2.	Count password inputs and inspect the second input type.	Two password inputs exist; confirmation input has type="password" .
TC_FR03_12	Verify the generated OTP length and format.	Boundary	Registered user.	OTP returned by a fresh request.	Request OTP and extract the digits displayed on the page.	Exactly six numeric digits are displayed.
TC_FR03_13	Reject an OTP longer than six digits.	Negative / Boundary	Registered user; OTP requested.	OTP 1234567 ; valid password.	Enter seven-digit OTP and submit reset.	OTP/token rejection.
TC_FR03_15	Accept a strong password at the lower boundary.	Positive / Boundary	Registered user; OTP requested.	Displayed OTP; eight-character Aa1!bbbb.	Submit valid OTP and password.	Success dialog and redirect to /login .
TC_FR03_16	Accept a strong password one above the lower boundary.	Positive / Boundary	Registered user; OTP requested.	Displayed OTP; nine-character Aa1!bbbb.	Submit valid OTP and password.	Success dialog and redirect to /login .
TC_FR03_17	Accept a strong password at	Positive / Boundary	Registered user; OTP requested.	Generated strong password of length 254.	Submit displayed OTP and	Success dialog and redirect to /login .

Test ID	Objective	Type	Preconditions	Input / strategy	Main steps	Expected result
	upper boundary minus one.				generated password.	
TC_FR03_18	Accept a strong password at the assumed upper boundary.	Positive / Boundary	Registered user; OTP requested.	Generated strong password of length 255.	Submit displayed OTP and generated password.	Success dialog and redirect to /login .
TC_FR03_19	Reject a password above the assumed upper boundary.	Negative / Boundary	Registered user; OTP requested.	Generated strong password of length 256.	Submit displayed OTP and generated password.	Password rejection without crash or truncation.

Pool B: FR-10 - Order State Machine

Each case creates a fresh order through the API and moves it through valid prerequisite transitions before testing the selected action. UI cases log in with the appropriate seeded account; direct API cases use an admin token or deliberately omit authorization.

Test cases:

Test ID	Objective	Actor	Start state	Channel	Expected HTTP	Expected persistent state
TC_FR10_01	Admin confirms a pending order.	Admin	pending	Admin UI	200	confirmed
TC_FR10_02	User cancels a pending order.	User	pending	User UI	200	canceled
TC_FR10_03	Admin ships a confirmed order.	Admin	confirmed	Admin UI	200	shipping
TC_FR10_04	User cancels a confirmed order.	User	confirmed	User UI	200	canceled
TC_FR10_05	Admin delivers a shipping order.	Admin	shipping	Admin UI	200	delivered
TC_FR10_06	Admin cancels a shipping order.	Admin	shipping	Admin UI	200	canceled
TC_FR10_07	User is prevented from canceling a shipping order.	User	shipping	User UI	400	Remains shipping
TC_FR10_08	Prevent skipping directly from pending to shipping.	Admin	pending	API	400	Remains pending
TC_FR10_09	Prevent moving backward from confirmed to pending.	Admin	confirmed	API	400	Remains confirmed

Test ID	Objective	Actor	Start state	Channel	Expected HTTP	Expected persistent state
TC_FR10_10	Prevent changing a delivered final-state order to canceled.	Admin	delivered	API	400	Remains delivered
TC_FR10_11	Prevent changing a canceled final-state order to pending.	Admin	canceled	API	400	Remains canceled
TC_FR10_12	Prevent changing a canceled final-state order to delivered.	Admin	canceled	Admin UI	400	Remains canceled
TC_FR10_13	Reject an unauthenticated order-state update.	Unauthenticated	pending	API without token	401	Remains pending

Pool C: FR-15 - Product CRUD

Every UI scenario logs in as `admin@eshop.com`. Unique product names include the test ID and browser to prevent collisions. Fixture setup and cleanup use an admin token, deterministic names are pre-cleaned, and deletion is verified after each case. Separate API cases check unauthenticated and regular-user authorization.

Test cases:

Test ID	Objective	Type	Input / condition	Expected result
TC_FR15_01	Create a valid product.	Positive	Valid name, price <code>25000000</code> , first category.	Product is created and appears in the table and API result.
TC_FR15_02	Reject an empty product name.	Negative	Name length 0; valid price/category.	No product is created.
TC_FR15_03	Reject a product name longer than 255 characters.	Negative / Boundary	Name length 256; valid price/category.	No product is created.
TC_FR15_04	Reject a nonnumeric price.	Negative	Price input <code>abc</code> .	Invalid product is not persisted.
TC_FR15_05	Reject price zero.	Negative / Boundary	Valid name/category; price <code>0</code> .	No product is created.
TC_FR15_06	Require an explicit category choice.	Negative / UI	Inspect category select with no category.	Empty option exists and the select is required.
TC_FR15_07	Prevent selection of a nonexistent category.	Negative / UI	Category ID <code>99999</code> .	No such option exists and it cannot be selected.
TC_FR15_08	Reject product name length zero at LB-1.	Negative / Boundary	Name length 0; price <code>100000</code> .	No product is created.
TC_FR15_09	Accept product name length one at LB.	Positive / Boundary	Generated one-character name; price <code>100000</code> .	Product is created.
TC_FR15_10	Accept product name length two at LB+1.	Positive / Boundary	Generated two-character name; price <code>100000</code> .	Product is created.

Test ID	Objective	Type	Input / condition	Expected result
TC_FR15_11	Accept product name length 254 at UB-1.	Positive / Boundary	Generated 254-character name.	Product is created.
TC_FR15_12	Accept product name length 255 at UB.	Positive / Boundary	Generated 255-character name.	Product is created.
TC_FR15_13	Reject product name length 256 at UB+1.	Negative / Boundary	Generated 256-character name.	No product is created.
TC_FR15_14	Reject price zero at LB-1.	Negative / Boundary	Valid name/category; price 0 .	No product is created.
TC_FR15_15	Accept price one at LB.	Positive / Boundary	Valid name/category; price 1 .	Product is created.
TC_FR15_16	Accept price two at LB+1.	Positive / Boundary	Valid name/category; price 2 .	Product is created.
TC_FR15_17	View a uniquely created product.	Positive / Read	API fixture with price 100000 .	Exact row is visible with correct name and price.
TC_FR15_18	Update only the selected product.	Positive / Update	Edit one fixture's name.	Exactly one row and one API record use the updated name; other rows are unchanged.
TC_FR15_19	Cancel product editing.	Positive / Update	Change a fixture's name, then cancel.	Form resets and stored/visible name remains unchanged.
TC_FR15_20	Delete the selected product.	Positive / Delete	Delete a uniquely identified fixture row.	DELETE returns 200; row and API record disappear.
TC_FR15_21	Reject unauthenticated product creation.	Negative / Authorization	POST a valid product without a token.	HTTP 401; no product is created.
TC_FR15_22	Reject product update by a regular user.	Negative / Authorization	PUT a fixture using the seeded customer token.	HTTP 403; the original product remains unchanged.
TC_FR15_23	Reject product deletion by a regular user.	Negative / Authorization	DELETE a fixture using the seeded customer token.	HTTP 403; the product remains stored.

Execution

Environment and method

Component	Value
Backend API	http://localhost:3000
Customer frontend	http://localhost:5173
Admin frontend	http://localhost:5174
Automation	Playwright 1.62.0 with TypeScript
Browsers	Chromium, Firefox, WebKit

The `run-feature.cjs` runner executes one feature on one browser and stores the report under `playwright/test-report/<feature>/<browser>`. All nine required feature-browser combinations were rerun sequentially and completed on 2026-08-07. A nonzero Playwright exit code represents failed assertions against the SUT; the HTML report is still generated.

Overall execution summary

Feature	Unique automated cases	Browser executions	Passed	Failed	Browser runs / HTML reports
FR-03	17	51	21	30	3
FR-10	13	39	30	9	3
FR-15	23	69	40	29	3
Total	53	159	91	68	9

HTML reports

Feature	Chromium	Firefox	WebKit
FR-03	Open report	Open report	Open report
FR-10	Open report	Open report	Open report
FR-15	Open report	Open report	Open report

FR-03 execution results

Test ID	Chromium	Firefox	WebKit
TC_FR03_01	FAILED	FAILED	FAILED
TC_FR03_02	PASSED	PASSED	PASSED
TC_FR03_03	FAILED	FAILED	FAILED
TC_FR03_04	FAILED	FAILED	FAILED
TC_FR03_05	PASSED	PASSED	PASSED
TC_FR03_06	PASSED	PASSED	PASSED
TC_FR03_07	PASSED	PASSED	PASSED
TC_FR03_08	PASSED	PASSED	PASSED
TC_FR03_09	PASSED	PASSED	PASSED
TC_FR03_10	FAILED	FAILED	FAILED
TC_FR03_12	FAILED	FAILED	FAILED
TC_FR03_13	FAILED	FAILED	FAILED
TC_FR03_15	FAILED	FAILED	FAILED
TC_FR03_16	FAILED	FAILED	FAILED
TC_FR03_17	FAILED	FAILED	FAILED
TC_FR03_18	FAILED	FAILED	FAILED
TC_FR03_19	PASSED	PASSED	PASSED
Total	7 passed / 10 failed	7 passed / 10 failed	7 passed / 10 failed

Observed failed-result groups:

- TC_FR03_01 and TC_FR03_15 through TC_FR03_18: valid strong passwords were rejected with the weak-password dialog, so no successful reset or login redirect occurred.

- TC_FR03_03, TC_FR03_04, and TC_FR03_13: OTP rejection could not reach its intended oracle because the frontend password validator rejected the otherwise valid password first.
- TC_FR03_10: only one password input existed instead of the required new-password and confirmation-password inputs.
- TC_FR03_12: the generated OTP contained four digits instead of six.

Assertion patterns used by FR-03:

- `toBeVisible()` for forms, inputs, and status messages.
- `toContain()` and `toContainText()` for dialog and page content.
- `toMatch()` for success/error message patterns and six-digit OTP format.
- `toHaveCount()` for confirmation-password input count.
- `toHaveAttribute()` for password input type.
- `toHaveURL()` for successful redirect and rejected-reset location.

FR-10 execution results

Test ID	Chromium	Firefox	WebKit
TC_FR10_01	PASSED	PASSED	PASSED
TC_FR10_02	PASSED	PASSED	PASSED
TC_FR10_03	PASSED	PASSED	PASSED
TC_FR10_04	PASSED	PASSED	PASSED
TC_FR10_05	PASSED	PASSED	PASSED
TC_FR10_06	FAILED	FAILED	FAILED
TC_FR10_07	FAILED	FAILED	FAILED
TC_FR10_08	PASSED	PASSED	PASSED
TC_FR10_09	PASSED	PASSED	PASSED
TC_FR10_10	PASSED	PASSED	PASSED
TC_FR10_11	PASSED	PASSED	PASSED
TC_FR10_12	FAILED	FAILED	FAILED
TC_FR10_13	PASSED	PASSED	PASSED
Total	10 passed / 3 failed	10 passed / 3 failed	10 passed / 3 failed

Observed failed-result groups:

- TC_FR10_06: the admin shipping-order row exposed only the delivery action; the required cancel action was absent and the order remained **shipping**.
- TC_FR10_07: a user could cancel a shipping order; the UI exposed the action, the request returned 200 instead of 400, and the stored state became **canceled**.
- TC_FR10_12: a canceled final-state order could be changed to **delivered**; the UI exposed the action, the request returned 200 instead of 400, and the stored state changed.

Assertion patterns used by FR-10:

- `toBe()` for HTTP response codes, button counts, and persisted state values.
- `toBeVisible()` for authenticated UI and exact order rows/actions.
- `toHaveURL()` for successful customer login.
- `toHaveCount()` for allowed or forbidden action-control counts.
- `toContainText()` for the visible localized order status.
- `expect.poll(...).toBe()` for asynchronous persisted-state transitions.

- `expect.soft(...)` for collecting related UI, HTTP, and state evidence in one failing case where continuation is safe.

FR-15 execution results

Test ID	Chromium	Firefox	WebKit
TC_FR15_01	PASSED	PASSED	PASSED
TC_FR15_02	PASSED	PASSED	PASSED
TC_FR15_03	FAILED	FAILED	FAILED
TC_FR15_04	FAILED	PASSED	FAILED
TC_FR15_05	FAILED	FAILED	FAILED
TC_FR15_06	FAILED	FAILED	FAILED
TC_FR15_07	PASSED	PASSED	PASSED
TC_FR15_08	PASSED	PASSED	PASSED
TC_FR15_09	PASSED	PASSED	PASSED
TC_FR15_10	PASSED	PASSED	PASSED
TC_FR15_11	PASSED	PASSED	PASSED
TC_FR15_12	PASSED	PASSED	PASSED
TC_FR15_13	FAILED	FAILED	FAILED
TC_FR15_14	FAILED	FAILED	FAILED
TC_FR15_15	PASSED	PASSED	PASSED
TC_FR15_16	PASSED	PASSED	PASSED
TC_FR15_17	PASSED	PASSED	PASSED
TC_FR15_18	FAILED	FAILED	FAILED
TC_FR15_19	PASSED	PASSED	PASSED
TC_FR15_20	PASSED	PASSED	PASSED
TC_FR15_21	FAILED	FAILED	FAILED
TC_FR15_22	FAILED	FAILED	FAILED
TC_FR15_23	FAILED	FAILED	FAILED
Total	13 passed / 10 failed	14 passed / 9 failed	13 passed / 10 failed

Observed failed-result groups:

- TC_FR15_03 and TC_FR15_13: product names of 256 characters were accepted and persisted instead of being rejected above the 255-character limit.
- TC_FR15_04: Chromium and WebKit submitted the form path and the SUT persisted a product with an empty price; Firefox's number input blocked the nonnumeric entry, so no product was persisted in that browser.
- TC_FR15_05 and TC_FR15_14: price 0 was accepted and persisted instead of being rejected as nonpositive.
- TC_FR15_06: the category control had no empty option and was not marked required; it preselected a category, preventing an explicit no-category choice.
- TC_FR15_18: after editing one product, six visible table rows displayed the updated name instead of exactly one, although the API contained one updated product record.
- TC_FR15_21: unauthenticated product creation returned 200 and persisted the product instead of returning 401.

- TC_FR15_22: a regular customer token could update a product; the API returned 200 and changed the stored record instead of returning 403.
- TC_FR15_23: a regular customer token could delete a product; the API returned 200 and removed the record instead of returning 403.

Assertion patterns used by FR-15:

- `toBeVisible()` for the admin product form and exact table rows.
- `toContainText()` for product name, price, and unchanged baseline products.
- `toHaveCount()` and `toHaveLength()` for table rows, stored products, deletion, and isolation.
- `toHaveAttribute()` for required-field validation.
- `toHaveValue()` and `not.toHaveValue()` for form reset and invalid category selection.
- `toBe()` for HTTP response codes, persisted price/category values, and native-request counts.
- `toBeGreaterThanOrEqual()` and `toBeLessThan()` for the expected 4xx rejection range.
- `expect.soft(...)` for preserving related authorization or validation evidence when safe.

Task 1 result summary

- Three selected features were automated with external CSV data and TypeScript Playwright specifications.
- Every feature contains at least 12 automated cases and at least three distinct assertion patterns.
- All three features ran on Chromium, Firefox, and WebKit, producing nine separate HTML reports.
- All 53 unique cases completed on all three browsers, for 159 executed browser instances.
- The final result was 91 passed and 68 failed assertions against the current SUT.
- Failed results were retained with Playwright screenshots, videos, trace archives, and error-context attachments according to the configured failure-evidence policy.

Human review and correction of the AI-generated scripts (AI analysis)

Area	Problem in the AI-generated version	My human correction	Why the AI likely missed it	Effect of the correction
FR-10 TC_FR10_06	The shipping-to-canceled branch only counted buttons, read the unchanged state, and returned. It never clicked Cancel or observed a response.	I removed the early return, located the exact row-scoped <code>Hủy</code> action, clicked it, required the exact PUT response, polled persisted state, and checked the visible status.	The generated control flow looked structurally similar to other state cases, so the model did not verify that the testcase's main action was actually executed.	TC_FR10_06 now fails for the real SUT problem: the shipping row has no required <code>Hủy</code> action. It would also execute correctly if the button were added later.
FR-03 password rules	All invalid-password rows accepted the same generic weak-password dialog as sufficient evidence. This could not prove whether the intended length, uppercase, lowercase, digit, special-character, or maximum-length rule caused rejection.	I added a rule-specific <code>Requirement</code> column and Playwright annotation for every row. I also added an <code>oracle-limitation</code> annotation explaining that the current SUT exposes only generic rejection evidence.	The SUT has one generic message and no validation rule code. The model mapped every negative row to the only visible message without distinguishing testcase-level evidence from generic rejection.	The report no longer overclaims rule-specific proof. The limitation is visible in each affected test result.
FR-15 response synchronization	Product creation waited only 1.5 seconds and converted every timeout	I removed the swallowed timeout. Valid forms wait for the exact POST	The AI used a short timeout as a convenient way to	The create tests now synchronize deterministically and

Area	Problem in the AI-generated version	My human correction	Why the AI likely missed it	Effect of the correction
	to <code>null</code> . This made “no request,” “slow request,” and “wrong predicate” indistinguishable and allowed a race with the API state query.	response; natively invalid forms attach a request listener and verify that no POST is sent.	support both browser-side and server-side rejection in one branch, but did not model their different observable outcomes.	preserve the actual rejection mechanism.
FR-15 fixture isolation	Cleanup ignored DELETE status, did not verify disappearance, and did not remove leftovers before a rerun. Fixture API calls also worked only because the current backend failed to protect mutation routes.	I authenticated setup/cleanup as admin, pre-cleaned deterministic names, required successful deletion, queried the API afterward, and retained <code>finally</code> cleanup.	Stateful database effects are difficult to infer from one isolated testcase, especially when the current insecure API still returns successful responses without authorization.	Interrupted or repeated runs are less likely to create false results. The final check found zero HW04 FR-15 fixture products remaining.
Selectors	Navigation used <code>li:nth(...)</code> ; edit/delete/order actions used <code>first()</code> , <code>nth(1)</code> , or calculated indexes; customer login used the first two generic text inputs.	I used exact visible menu text, exact row scoping, and accessible action names such as <code>Sửa</code> , <code>Xóa</code> , <code>Hủy</code> , and <code>Hủy đơn</code> . Login inputs are scoped from their visible label containers because the SUT labels are not programmatically associated.	The AI copied the current DOM order because it was immediately available. It did not sufficiently consider menu insertion, action reordering, localization, or destructive clicks.	The selectors express user intent and are less likely to click the wrong record or action after layout changes.
FR-15 creation oracle	A successful create required only response 200, at least one name match, and a visible name. The CSV category was not explicitly selected. Price, category, uniqueness, and form reset were not proved.	I explicitly apply <code>Category=first</code> and assert exactly one record, persisted price, persisted <code>category_id</code> , visible name/price, and cleared form inputs.	The initial prompt emphasized converting testcase rows into runnable scripts, and the AI stopped at the most obvious success indicators instead of tracing every important input to a stored and visible output.	A partial or duplicated product can no longer satisfy the positive create oracle.
FR-15 validation evidence	The two missing-category assertions were hard assertions, so failure of the empty-option check prevented collection of the missing <code>required</code> attribute.	I changed both independent DOM checks to <code>expect.soft(...)</code> .	The model applied the normal fail-fast assertion pattern without considering that both read-only observations could safely be collected.	One execution now records both pieces of validation evidence.

Area	Problem in the AI-generated version	My human correction	Why the AI likely missed it	Effect of the correction
FR-15 authorization coverage	The draft tested admin UI behavior but had no negative authorization cases for Product CRUD.	I added TC_FR15_21 for unauthenticated create, TC_FR15_22 for regular-user update, and TC_FR15_23 for regular-user delete, including unchanged-state assertions.	The selected CRUD flow was framed mainly as an admin UI feature. Unless authorization is explicitly requested, a model can confuse successful admin navigation with server-side access control coverage.	The suite increased from 50 to 53 logical cases. All three new cases fail on every browser because the backend returns 200 and changes state instead of returning 401/403.

After fixing code:

Version	Logical cases	Browser executions	Passed	Failed	Interpretation
AI-generated draft	50	150	91	59	Runnable, but it contained incomplete or weak checks and omitted Product CRUD authorization.
Human-reviewed final version	53	159	91	68	Three authorization cases added; all nine added browser executions revealed genuine missing access control. Existing workflows and oracles were strengthened without changing correct expectations merely to make tests pass.

Task 2

[Youtube link](#)

Task 3

The skill should guide an agent through:

1. Reading a feature requirement and testcase table.
2. Producing an external data schema.
3. Inspecting the relevant UI for selectors.
4. Generating Playwright tests in small groups.
5. Configuring three browsers and report metadata.
6. Reviewing selectors, waits, assertions, isolation, and cleanup.
7. Classifying failures and producing a report checklist.

[Agent skill](#)

Appendices

Appendix A

[AI Audit Report](#)

[AI Critique](#)

Appendix B

Self-assessment & Test summary report